

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: INTERNET PROGRAMMING
COURSE CODE: CSA4305

COURSE OUTCOME COVERED

CO -2 : Apply CSS layout and styling techniques to create visually appealing and responsive web interfaces, and use JavaScript to enable interactive client-side behavior.
(L3)

Assessment Tool : Design Task

Weightage: 20%

QUESTIONS:

Total Marks: 20

Question 1: Responsive Navigation Bar Design - Marks(4)

Suppose that an e-commerce website requires a responsive navigation bar that adapts across desktop, tablet, and mobile devices.

- (a) Identify key components required in a navigation bar for an e-commerce website.
- (b) Design a responsive navigation bar using appropriate CSS techniques.
- (c) Demonstrate how the navigation adapts to smaller screens (e.g., hamburger menu).
- (d) Evaluate the usability and suggest improvements for better user experience.

Question 2: Product Hover Effect - Marks(4)

Suppose that an e-commerce website wants to enhance user interaction through visual effects on product items.

- (a) Identify suitable CSS properties for hover effects.
- (b) Suggest appropriate **CSS animation or transition** for product interaction.
- (c) Design a hover effect for a product card.
- (d) Evaluate performance and user experience considerations.

Question 3: Layout Selection (Flexbox vs Grid) - Marks(4)

Suppose that a dashboard layout needs to be designed for a web application.

- (a) Identify layout requirements of a dashboard (rows, columns, alignment).
- (b) Compare **Flexbox and Grid** based on layout needs.
- (c) Choose the most suitable layout technique and justify your choice.
- (d) Evaluate the scalability and responsiveness of the chosen approach.

Question 4: Mobile-First Blog Layout - Marks(4)

Suppose that a blog page must be designed following a mobile-first approach.

- (a) Identify key sections of a blog layout.
- (b) Design a mobile-first layout using CSS.
- (c) Demonstrate how the layout scales for larger screens using media queries.
- (d) Evaluate accessibility and readability of the design.

Question 5: JavaScript Event Handling for Dynamic Menu - Marks(4)

Suppose that a website includes a dynamic menu that responds to user interactions.

- (a) Identify appropriate JavaScript events for menu interaction.
- (b) Design event handling logic for menu toggle functionality.
- (c) Implement interaction for user actions (click/hover).
- (d) Evaluate performance and suggest improvements.

Code of Conduct:

I P Madhavi/192372133(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:Madhavi

RUBRICS FOR CALCULATION:

Criteria	Weightage (%)	Level 4 – Exemplary (4)	Level 3 – Proficient (3)	Level 2 – Developing (2)	Level 1 – Beginning (1)
Understanding of Concept	25	Demonstrates complete understanding of design requirements and concepts	Good understanding with minor gaps	Basic understanding with some errors	Poor understanding or incorrect concepts
Design /	30	Well-structured,	Correct	Basic	Incorrect or

Implementatio		efficient, and	implementatio	implementatio	incomplete
n		responsive design implementation	n with minor issues	n with inconsistencies	implementatio n
Use of Tools (CSS/JS)	20	Effective and advanced use of tools/technique s	Appropriate use of tools	Limited or inconsistent use	Incorrect or minimal use
Analysis & Evaluation	15	Insightful analysis with strong justification and optimization	Good analysis with justification	Basic analysis with limited depth	Poor or no analysis
Documentation / Clarity	10	Clear, wellstructured, and properly presented solution	Mostly clear with minor issues	Basic clarity, missing details	Poor or unclear documentation

Question 1: Responsive Navigation Bar Design

(a) Identify key components required in a navigation bar for an e-commerce website.

The important components of an e-commerce navigation bar are:

- Company logo
- Home link
- Product categories
- Search bar
- Shopping cart icon
- Wishlist icon
- User login/profile
- Contact page
- Responsive hamburger menu for mobile devices

(b) Design a responsive navigation bar using appropriate CSS techniques.

HTML

```

<!DOCTYPE html>
<html>
<head>
<title>Responsive Navbar</title>
<link rel="stylesheet" href="style.css">
</head>
<body>

<nav class="navbar">
  <div class="logo">ShopEasy</div>

```

```
<ul class="nav-links">
  <li><a href="#">Home</a></li>
  <li><a href="#">Products</a></li>
  <li><a href="#">Categories</a></li>
  <li><a href="#">Cart</a></li>
  <li><a href="#">Contact</a></li>
</ul>

<div class="menu">&#9776;</div>
</nav>
```

```
</body>
</html>
```

CSS

```
*{
margin:0;
padding:0;
box-sizing:border-box;
}
```

```
.navbar{
display:flex;
justify-content:space-between;
align-items:center;
padding:15px 20px;
background:#333;
color:white;
}
```

```
.logo{
font-size:24px;
font-weight:bold;
}
```

```
.nav-links{
display:flex;
list-style:none;
}
```

```
.nav-links li{
margin:0 15px;
}
```

```
.nav-links a{
color:white;
```

```
text-decoration:none;
}
```

```
.menu{
display:none;
font-size:30px;
cursor:pointer;
}
```

(c) Demonstrate how the navigation adapts to smaller screens (Hamburger Menu).

```
@media(max-width:768px){
```

```
.nav-links{
display:none;
flex-direction:column;
width:100%;
background:#333;
}
```

```
.menu{
display:block;
}
```

```
.nav-links.active{
display:flex;
}
}
```

```
const menu=document.querySelector(".menu");
const nav=document.querySelector(".nav-links");
```

```
menu.onclick=()=>{
nav.classList.toggle("active");
};
```

(d) Evaluate the usability and suggest improvements for better user experience.

- Mobile-friendly navigation improves accessibility.
- Hamburger menu saves screen space.
- Search bar enables faster product discovery.
- Large buttons improve touch interaction.
- Highlighting the active page enhances navigation.
- Sticky navigation bar allows quick access while scrolling.

Question 2: Product Hover Effect

(a) Identify suitable CSS properties for hover effects.

Suitable CSS properties include:

- transform
- transition
- box-shadow
- opacity
- background-color
- color
- border-radius
- filter

(b) Suggest appropriate CSS animation or transition for product interaction.

A smooth transition with scaling and shadow effect provides better interaction.

transition:0.4s ease;

transform:scale(1.05);

box-shadow:0 10px 20px rgba(0,0,0,0.3);

(c) Design a hover effect for a product card.

HTML

```
<div class="card">
```

```

```

```
<h3>Wireless Headphone</h3>
```

```
<p>$120</p>
```

```
<button>Buy Now</button>
```

```
</div>
```

CSS

```
.card{
```

```
width:250px;
```

```
padding:20px;
```

```
text-align:center;
```

```
border:1px solid gray;
```

```
transition:0.4s;
```

```
}
```

```
.card:hover{
```

```
transform:scale(1.05);
```

```
box-shadow:0 8px 20px gray;
```

```
}
```

```
button:hover{
```

```
background:green;
```

```
color:white;
```

```
}
```

(d) Evaluate performance and user experience considerations.

- Smooth transitions improve visual appeal.
- Avoid excessive animations that reduce performance.

- Use GPU-accelerated properties like transform.
- Ensure hover effects do not distract users.
- Keep animations under 0.5 seconds.

Question 3: Layout Selection (Flexbox vs Grid)

(a) Identify layout requirements of a dashboard (rows, columns, alignment).

A dashboard generally contains:

- Header
- Sidebar
- Main content
- Widgets
- Charts
- Footer
- Proper spacing and alignment
- Responsive layout

(b) Compare Flexbox and Grid based on layout needs.

Feature	Flexbox	Grid
Layout Type	One-dimensional	Two-dimensional
Direction	Row or Column	Rows and Columns
Alignment	Excellent	Excellent
Complex Layout	Limited	Best
Responsive	Good	Excellent

(c) Choose the most suitable layout technique and justify your choice.

CSS Grid is the most suitable because:

- Supports rows and columns simultaneously.
- Easy placement of dashboard widgets.
- Better control over layout.
- Easier responsive design.

Example:

```
.container{
display:grid;
grid-template-columns:250px auto;
grid-template-rows:80px auto;
gap:15px;
}
```

(d) Evaluate the scalability and responsiveness of the chosen approach.

- Grid adapts easily to different screen sizes.
- Media queries can rearrange columns.

- Easy to add new dashboard widgets.
- Maintains consistent spacing and alignment.

Question 4: Mobile-First Blog Layout

(a) Identify key sections of a blog layout.

The main sections include:

- Header
- Navigation
- Featured Image
- Blog Content
- Sidebar
- Author Information
- Comments
- Footer

(b) Design a mobile-first layout using CSS.

HTML

```
<div class="container">
<header>My Blog</header>

<main>
<h2>Blog Title</h2>
<p>Blog content goes here.</p>
</main>

<footer>Copyright 2026</footer>
</div>
```

CSS

```
.container{
display:flex;
flex-direction:column;
padding:15px;
}

header,footer{
background:#333;
color:white;
padding:15px;
text-align:center;
}
```

(c) Demonstrate how the layout scales for larger screens using media queries.

```
@media(min-width:768px){
```



```
.container{  
max-width:1000px;  
margin:auto;  
}
```

```
main{  
display:flex;  
gap:20px;  
}  
}
```

(d) Evaluate accessibility and readability of the design.

- Responsive font sizes improve readability.
- Proper color contrast enhances accessibility.
- Larger buttons improve touch interaction.
- Semantic HTML benefits screen readers.
- Consistent spacing improves user experience.

Question 5: JavaScript Event Handling for Dynamic Menu

(a) Identify appropriate JavaScript events for menu interaction.

Common JavaScript events include:

- click
- mouseover
- mouseout
- keydown
- touchstart
- focus
- blur

(b) Design event handling logic for menu toggle functionality.

```
const menu=document.querySelector(".menu");  
const nav=document.querySelector(".nav-links");
```

```
menu.addEventListener("click",function(){  
nav.classList.toggle("active");  
});
```

(c) Implement interaction for user actions (click/hover).

HTML

```
<div class="menu">☰</div>
```

```
<ul class="nav-links">  
<li>Home</li>  
<li>Products</li>  
<li>About</li>  
<li>Contact</li>  
</ul>
```

CSS

```
.nav-links{  
display:none;  
}
```

```
.nav-links.active{  
display:block;  
}
```

```
.nav-links li:hover{  
background:#555;  
color:white;  
cursor:pointer;  
}
```

JavaScript

```
const menu=document.querySelector(".menu");  
const nav=document.querySelector(".nav-links");
```

```
menu.onclick=function(){  
nav.classList.toggle("active");  
};
```

(d) Evaluate performance and suggest improvements.

- Use `addEventListener()` instead of inline event handlers.
- Minimize DOM manipulation for better performance.
- Debounce frequent events like `resize` or `scroll`.
- Ensure keyboard accessibility for navigation.
- Optimize JavaScript execution to improve responsiveness.